JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Class Package

java.lang.Object
    java.lang.Package

All Implemented Interfaces:

AnnotatedElement

```
public class Package
extends Object
implements AnnotatedElement
```

Package objects contain version information about the implementation and specification of a Java package. This versioning information is retrieved and made available by the ClassLoader instance that loaded the class(es). Typically, it is stored in the manifest that is distributed with the classes.

The set of classes that make up the package may implement a particular specification and if so the specification title, version number, and vendor strings identify that specification. An application can ask if the package is compatible with a particular version, see the isCompatibleWith method for details.

Specification version numbers use a syntax that consists of nonnegative decimal integers separated by periods ".", for example "2.0" or "1.2.3.4.5.6.7". This allows an extensible number to be used to represent major, minor, micro, etc. versions. The version specification is described by the following formal grammar:

SpecificationVersion:

Digits RefinedVersion$_{opt}$

RefinedVersion:

. Digits

. Digits RefinedVersion

Digits:

Digit

Digits

Digit:

any character for which `Character.isDigit(char)` returns true, e.g. 0, 1, 2, ...

The implementation title, version, and vendor strings identify an implementation and are made available conveniently to enable accurate reporting of the packages involved when a problem occurs. The contents all three implementation strings are vendor specific. The implementation version strings have no specified syntax and should only be compared for equality with desired version identifiers.

Within each `ClassLoader` instance all classes from the same java package have the same Package object. The static methods allow a package to be found by name or the set of all packages known to the current class loader to be found.

See Also:

`ClassLoader.definePackage(java.lang.String, java.lang.String, java.lang.String, java.lang.String, java.lang.String, java.lang.String, java.lang.String, java.net.URL)`

Method Summary

Concrete Methods

Modifier and Type	Method	De
<A extends **Annotation**> A	**getAnnotation**(**Class**<A> annotationClass)	R el ar th ty ar *p* n
Annotation[]	**getAnnotations**()	R ar th o
<A extends **Annotation**> A[]	**getAnnotationsByType**(**Class**<A> annotationClass)	R ar th *a* th
	getDeclaredAnnotation(**Class**<A> annotationClass)	

<A extends **Annotation**> A		R el ar th ty ar *d p* n
Annotation[]	**getDeclaredAnnotations**()	R ar th *p* el
<A extends **Annotation**> A[]	**getDeclaredAnnotationsByType**(**Class**<A> annotationClass)	R el ar fo ty ar ei *p in p*
String	**getImplementationTitle**()	R of
String	**getImplementationVendor**()	R na or ve co pr in
String	**getImplementationVersion**()	R ve in
String	**getName**()	R na pa
static **Package**	**getPackage**(**String** name)	F by ca

		C in
static **Package**[]	**getPackages**()	G pa cu kn ca C in
String	**getSpecificationTitle**()	R of sp th pa in
String	**getSpecificationVendor**()	R na of ve co ow m sp th in pa
String	**getSpecificationVersion**()	R ve of sp th pa in
int	**hashCode**()	R co fr pa
boolean	**isAnnotationPresent**(**Class**<? extends **Annotation**> annotationClass)	R an fo ty of el

boolean	**isCompatibleWith**(**String** desired)	C p s v d
boolean	**isSealed**()	R th se
boolean	**isSealed**(**URL** url)	R th se re sp sc
String	**toString**()	R st re o

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Detail

getName

```
public String getName()
```

Return the name of this package.

Returns:

The fully-qualified name of this package as defined in section 6.5.3 of *The Java™ Language Specification*, for example, java.lang

getSpecificationTitle

```
public String getSpecificationTitle()
```

Return the title of the specification that this package implements.

Returns:

the specification title, null is returned if it is not known.

getSpecificationVersion

```
public String getSpecificationVersion()
```

Returns the version number of the specification that this package implements. This version string must be a sequence of nonnegative decimal integers separated by "."'s and may have leading zeros. When version strings are compared the most significant numbers are compared.

Returns:

the specification version, null is returned if it is not known.

getSpecificationVendor

```
public String getSpecificationVendor()
```

Return the name of the organization, vendor, or company that owns and maintains the specification of the classes that implement this package.

Returns:

the specification vendor, null is returned if it is not known.

getImplementationTitle

```
public String getImplementationTitle()
```

Return the title of this package.

Returns:

the title of the implementation, null is returned if it is not known.

getImplementationVersion

```
public String getImplementationVersion()
```

Return the version of this implementation. It consists of any string assigned by the vendor of this implementation and does not have any particular syntax specified or expected by the Java runtime. It may be compared for equality with other package version strings used for this implementation by this vendor for this package.

Returns:

the version of the implementation, null is returned if it is not known.

getImplementationVendor

```
public String getImplementationVendor()
```

Returns the name of the organization, vendor or company that provided this implementation.

Returns:

the vendor that implemented this package..

isSealed

```
public boolean isSealed()
```

Returns true if this package is sealed.

Returns:

true if the package is sealed, false otherwise

isSealed

```
public boolean isSealed(URL url)
```

Returns true if this package is sealed with respect to the specified code source url.

Parameters:

url - the code source url

Returns:

true if this package is sealed with respect to url

isCompatibleWith

```
public boolean isCompatibleWith(String desired)
                         throws NumberFormatException
```

Compare this package's specification version with a desired version. It returns true if this packages specification version number is greater than or equal to the desired version number.

Version numbers are compared by sequentially comparing corresponding components of the desired and specification strings. Each component is

converted as a decimal integer and the values compared. If the specification value is greater than the desired value true is returned. If the value is less false is returned. If the values are equal the period is skipped and the next pair of components is compared.

Parameters:

`desired - the version string of the desired version.`

Returns:

`true if this package's version number is greater than or equal to the desired version number`

Throws:

`NumberFormatException - if the desired or current version is not of the correct dotted form.`

getPackage

```
public static Package getPackage(String name)
```

Find a package by name in the callers `ClassLoader` instance. The callers `ClassLoader` instance is used to find the package instance corresponding to the named class. If the callers `ClassLoader` instance is null then the set of packages loaded by the system `ClassLoader` instance is searched to find the named package.

Packages have attributes for versions and specifications only if the class loader created the package instance with the appropriate attributes. Typically, those attributes are defined in the manifests that accompany the classes.

Parameters:

`name - a package name, for example, java.lang.`

Returns:

`the package of the requested name. It may be null if no package information is available from the archive or codebase.`

getPackages

```
public static Package[] getPackages()
```

Get all the packages currently known for the caller's `ClassLoader` instance. Those packages correspond to classes loaded via or accessible by name to that `ClassLoader` instance. If the caller's `ClassLoader` instance is the bootstrap `ClassLoader` instance, which may be represented by `null` in some implementations, only packages corresponding to classes loaded by the bootstrap `ClassLoader` instance will be returned.

Returns:

a new array of packages known to the callers ClassLoader instance. An zero length array is returned if none are known.

hashCode

```
public int hashCode()
```

Return the hash code computed from the package name.

Overrides:

`hashCode` in class `Object`

Returns:

the hash code computed from the package name.

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

toString

```
public String toString()
```

Returns the string representation of this Package. Its value is the string "package " and the package name. If the package title is defined it is appended. If the package version is defined it is appended.

Overrides:

`toString` in class `Object`

Returns:

the string representation of the package.

getAnnotation

```
public <A extends Annotation> A getAnnotation(Class<A> annotationClass)
```

Description copied from interface: `AnnotatedElement`

Returns this element's annotation for the specified type if such an annotation is *present*, else null.

Specified by:

`getAnnotation` in interface `AnnotatedElement`

Type Parameters:

A - the type of the annotation to query for and return if present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if present on this element, else null

Throws:

NullPointerException - if the given annotation class is null

Since:

1.5

isAnnotationPresent

```
public boolean isAnnotationPresent(Class<? extends Annotation> annotationClass)
```

Returns true if an annotation for the specified type is *present* on this element, else false. This method is designed primarily for convenient access to marker annotations.

The truth value returned by this method is equivalent to: `getAnnotation(annotationClass) != null`

The body of the default method is specified to be the code above.

Specified by:

isAnnotationPresent in interface AnnotatedElement

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

true if an annotation for the specified annotation type is present on this element, else false

Throws:

NullPointerException - if the given annotation class is null

Since:

1.5

getAnnotationsByType

```
public <A extends Annotation> A[] getAnnotationsByType(Class<A> annotationClass)
```

Description copied from interface: AnnotatedElement

Returns annotations that are *associated* with this element. If there are no annotations *associated* with this element, the return value is an array of length 0. The difference between this method and `AnnotatedElement.getAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS 9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getAnnotationsByType` in interface `AnnotatedElement`

Type Parameters:

`A - the type of the annotation to query for and return if present`

Parameters:

`annotationClass - the Class object corresponding to the annotation type`

Returns:

`all this element's annotations for the specified annotation type if associated with this element, else an array of length zero`

Throws:

`NullPointerException - if the given annotation class is null`

Since:

`1.8`

getAnnotations

```
public Annotation[] getAnnotations()
```

Description copied from interface: `AnnotatedElement`

Returns annotations that are *present* on this element. If there are no annotations *present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getAnnotations` in interface `AnnotatedElement`

Returns:

`annotations present on this element`

Since:

`1.5`

getDeclaredAnnotation

```
public <A extends Annotation> A getDeclaredAnnotation(Class<A> annotationClass)
```

Description copied from interface: AnnotatedElement

Returns this element's annotation for the specified type if such an annotation is *directly present*, else null. This method ignores inherited annotations. (Returns null if no annotations are directly present on this element.)

Specified by:

getDeclaredAnnotation in interface AnnotatedElement

Type Parameters:

A - the type of the annotation to query for and return if directly present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if directly present on this element, else null

Throws:

NullPointerException - if the given annotation class is null

Since:

1.8

getDeclaredAnnotationsByType

```
public <A extends Annotation> A[] getDeclaredAnnotationsByType(Class<A> annotatio
```

Description copied from interface: AnnotatedElement

Returns this element's annotation(s) for the specified type if such annotations are either *directly present* or *indirectly present*. This method ignores inherited annotations. If there are no specified annotations directly or indirectly present on this element, the return value is an array of length 0. The difference between this method and AnnotatedElement.getDeclaredAnnotation(Class) is that this method detects if its argument is a *repeatable annotation type* (JLS 9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation if one is present. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

getDeclaredAnnotationsByType in interface AnnotatedElement

Type Parameters:

A - the type of the annotation to query for and return if directly or indirectly present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

all this element's annotations for the specified annotation type if directly or indirectly present on this element, else an array of length zero

Throws:

NullPointerException - if the given annotation class is null

Since:

1.8

getDeclaredAnnotations

```
public Annotation[] getDeclaredAnnotations()
```

Description copied from interface: AnnotatedElement

Returns annotations that are *directly present* on this element. This method ignores inherited annotations. If there are no annotations *directly present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

getDeclaredAnnotations in interface AnnotatedElement

Returns:

annotations directly present on this element

Since:

```
1.5
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.